Distilling Phi-3.5 into Pythia-70M for Command Parsing

CS3081 Cloud Computing Mini Project
Rishi Raj, Roll No. 230103019, BTech CSE, Semester 7, IIIT Manipur
9 November 2026

GitHub: https://github.com/Rishi-Ra-1j/Cloud_Project | Model Release: https://github.com/Rishi-Ra-1j/Cloud_Project/releases/tag/v1

Teacher: Phi-3.5-mini-instruct (3.8B, Ollama, temperature 0, seed 42). Student: EleutherAI/pythia-70m (70M, PyTorch + Hugging Face Transformers, CPU). Task: parse one short English command into exactly one JSON object with exactly four keys — **intent, date, time, place** — with eight intents (alarm_set, alarm_query, calendar_set, calendar_query, calendar_remove, weather_query, datetime_query, recommendation_events). Slot values are copied exactly as in the command, never converted; a slot not mentioned is **null**, never invented. All numbers in this report are measured: from score.py, kubectl, Locust, or the training logs. Where a planned measurement could not be completed, that is stated, not filled in.

Environment. Acer Aspire A715-51G laptop, Ubuntu Linux 6.17, 16 logical CPU cores, 6.7 GB RAM, Python 3.11.7, Docker 29.1.3. RAM was the binding constraint for the whole project: Ollama, training, and the k3d cluster were run sequentially, never together, and 11 GB of swap was added (pre-existing 4 GB /swap.img + new 8 GB /swapfile). GPU: GTX 1650, 4 GB VRAM. Software verified on the laptop: torch 2.15.0+cpu, transformers 5.22.2, datasets 5.1.0, fastapi 0.162.2, uvicorn 0.54.0, boto3 1.43.126, locust 2.47.0, kubectl v1.37.2, k3d v5.9.0 (k3s v1.35.5+k3s1), Ollama v0.40.15.

1 Data pipeline and teacher labelling (Task 1)

A single-node k3d cluster **cs3081** was created (node k3d-cs3081-server-0, Ready, v1.35.5+k3s1). SeaweedFS was deployed in namespace **data** (single pod seaweedfs-0, 1/1 Running, image chrislusf/seaweedfs:4.48, S3 on port 8333), with buckets **datasets** and **models**; *commands_pool.jsonl* (3,000 commands, 197,541 bytes) was uploaded to s3://datasets/. Two teacher workers ran at the same time in namespace **pipeline** (Jobs teacher-worker-0 and teacher-worker-1, SHARD_COUNT=2, sharding by line number % 2), each calling Ollama phi3.5 at host.k3d.internal:11434 with temperature 0 and seed 42, using the prompt in prompts/teacher_prompt.txt. Teacher replies were stored exactly as generated, never edited manually, as s3://datasets/teacher/shard-0.jsonl and shard-1.jsonl, 1,500 records each.

The first labelling attempt failed in a way that shaped the rest of the project. Both shards reached 50/1,500 quickly, then Ollama returned HTTP 500: one teacher reply generated 1,784 tokens over 2 minutes 20 seconds instead of stopping after one short JSON, blocking Ollama's single slot for the other worker. The fix was to cap generation (num_predict=128), add stop sequences, raise retries to 5, lower the timeout to 120 s, resume from the existing shard objects, and upload after every 50 records. The fixed run completed both shards, 1,500/1,500 each, with no errors, in about 5.5 hours wall time.

Cleaning (worker/clean.py) removed, in order, (a) invalid JSON, (b) wrong keys, (c) bad intent, and (d) a slot value that is not null and not in the command, plus duplicates keeping the smallest id. Counts, which sum exactly:

	Count
Input commands	3,000
(a) invalid JSON	264
(b) wrong keys	0
(c) bad intent	0
(d) slot not in command	43
Duplicates	43
Kept	2,650

264 + 0 + 0 + 43 + 43 + 2,650 = 3,000. The 264 invalid-JSON replies are the teacher rambling on some commands, the same behaviour that caused the 500 above. The cleaned file is s3://datasets/v1/train.jsonl (2,650 records), sha256 311026d87493c44df57cb413b7326779456b126825a529f2845d5787488df043. Training used only the first 1,500 of these, as the assignment requires.

2 Distillation and training (Task 2)

Training used training/config.yaml (seed 42, learning rate 5e-5, batch size 4, gradient accumulation 2, epochs 3, max sequence length 128, max 1,500 examples) and *make train* / *make train SUBSET=50*. Each example is the text *Command: <text> JSON: <teacher_raw>*. Two real bugs were found and fixed. First, the smoke test failed at 0/21 steps because tokenised examples have different lengths and DataCollatorForLanguageModeling did not pad them under transformers 5.22.2; a custom collator now pads input ids, attention mask, and labels (−100) to the longest example in the batch. Second, the smoke test then ran but with broken loss (1.691e+04, then 0, grad_norm NaN, 64.66 s/step): Pythia-70M is stored in float16, and training ran in fp16 on CPU with no GradScaler, so gradients overflowed and CPU fp16 math was very slow. Loading the model in float32 fixed it: smoke-test loss 1.716 then 0.6994, grad_norm 21.26/21.48, 17.58 s for 50 examples.

Model	Steps	Runtime	Train loss	Final loss
v1 (whole sequence)	564/564	712.6 s (11:52)	0.7789	0.5213 at epoch 2.981
v2 (masked, JSON only)	564/564	720.1 s (12:00)	0.1622	—
v2 retrain (fixed tokenisation)	564/564	669 s (11:09)	0.1622	—

The two losses are not directly comparable: v2's loss is averaged over JSON tokens only, v1's over prompt + JSON tokens. Models were saved to models/v1 and models/v2 with model.sha256 and uploaded to s3://models/v1/ and s3://models/v2/; v1 is also the GitHub Release asset (Task 6).

Sequence-level vs logit-based distillation. This project uses sequence-level distillation: the student is trained with cross-entropy on the teacher's chosen output sequence, the one JSON object the teacher actually generated. Logit-based distillation would instead match the teacher's per-token probability distribution with a KL loss. That is impossible here because Ollama's /api/generate returns text only — no logits, no log-probabilities, and no weights. One real limitation follows: the student inherits the teacher's silent, well-formed but wrong replies, which cleaning cannot detect, so the student's ceiling is the teacher's accuracy.

Evaluation (scripts/evaluate.py, greedy, max 128 new tokens, 100 validation commands, scored with tools/score.py). The first student evaluation scored 0.0% on every metric, not because the model was wrong but because score.py does strict json.loads on the whole prediction, and v1 emits a short prefix before the JSON. Extracting the first balanced JSON object from each prediction — the method used for the clean score below — recovers the real result. The 7.0% validity figure from a later re-evaluation is an extraction bug (first '{' to last '}' spans several JSON objects the model emits in 128 tokens) and is not a model score.

Model	Validity	Intent	Date	Time	Place	Full frame
Teacher (phi3.5)	86.0	73.0	76.0	74.0	64.0	37.0
Student v1 (clean)	93.0	78.0	71.0	75.0	68.0	35.0
Untrained pythia-70m	0.0	0.0	0.0	0.0	0.0	0.0

Student v1 full-frame accuracy is 35%, against the teacher's 37%: the bar for full marks is half the teacher's, 18.5%, so v1 passes it, and it beats the teacher on validity (93 vs 86) and intent (78 vs 73). Validity 93% is just below the 95% quality gate in score.py --min-validity 95, so the CI gate does not pass on v1 — that is stated, not hidden. The untrained base at 0% is the control showing training is what produced the result, and it is the deliberately failed CI gate run in Task 6.

v2 is a failed experiment, reported as such. The masked v2 (prompt labels −100, train only on JSON reply, EOS appended) evaluated at 0.0% on every metric: predictions were mostly empty strings. Diagnosis: the prompt ended with a trailing space, so the prompt tokens were not a prefix of the training text — the space merges with '{' into one token in the full text. Fixing that and retraining v2 (564 steps, 11:09, loss 0.1622) still gave 0.0%. A diagnostic run (scripts/test_one.py) on V001 shows why: v1 generates ' {' first (top-5 next token, logit 986), while v2 generates a newline then EOS, with ' {' not in its top 5. Masked training broke first-token prediction. v2 was therefore not deployed; v1 is the deploy model.

3 Service (Task 3)

service/app.py is FastAPI on port 8000: POST /parse returns only the four-key JSON with headers X-Model-Version and X-Served-By (pod hostname); GET /healthz returns 200; GET /readyz returns 503 until the model is loaded. At startup the model is downloaded from MODEL_URI (s3://models/v1) using S3 credentials from a Kubernetes Secret only, and output is validated against schema.json — exactly four keys, a valid intent, slots

string-or-null — otherwise HTTP 500 {"error": "invalid_model_output"}; FAULT_INJECT=1 forces that 500. The image (cs3081-parser:v1, python:3.11-slim, pinned CPU torch 2.15.0, transformers 5.22.2, no weights in the image) was built in 108 s.

Deployment initially failed: both pods CrashLoopBackOff with NoSuchBucket. The S3 URI parser in download_model split s3://models/v1 into bucket 'v1' instead of bucket 'models'. Fixing the parser (bucket=models, prefix=v1, image sha 9fce96c6) made the pods download all 7 model files and load v1. A second, smaller failure on the way was human: the fixed app.py landed in ~/Downloads as app (1).py, so the first rebuild used the old file and produced the same image sha; checking the sha caught it.

Verified end to end via the Ingress (host parse.local, port 8080):

POST /parse {"text": "set an alarm for seven am tomorrow"} → HTTP 200, X-Model-Version: v1, X-Served-By: parse-service-655569688-mkhh5, {"intent": "alarm_set", "date": "tomorrow", "time": "seven am", "place": null}

Peak memory from kubectl top pods during the load test was 724 Mi for one replica (the other 383 Mi, SeaweedFS 176 Mi), under the 1 GiB limit. A 50-sequential-request memory run was not completed in the time available; the top-pods figures above are the memory evidence. Single-request latency was 26.2 s (curl, one parse, 128 generated tokens on CPU) — slow, and it matters in Task 7.

4 Kubernetes deployment (Task 4)

Namespace app, Deployment parse-service (2 replicas, requests 500m CPU / 512 Mi, limits 1 CPU / 1 GiB, liveness /healthz, readiness /readyz with 60 s initial delay), Service (port 80 → 8000), Ingress (Traefik, host parse.local, path /parse), HPA (min 1, max 2, 70% CPU).

Load balancing. 10 parallel requests: 5 served by parse-service-655569688-f668b and 5 by parse-service-655569688-mkhh5 — 50/50, each above the 30% bar. A sequential 20-request run was abandoned because at ~26 s per parse it would take about 9 minutes.

Pod delete and cold start. Pod f668b was deleted at 09:28:32; the replacement dqqqx was 1/1 Ready at 09:30:14 — a cold start of 102 seconds, including the model download from SeaweedFS. The other pod stayed 1/1 throughout, so the service never went down. Only one cold-start measurement was taken in the time available; the assignment asks for three, and the median is therefore not claimed.

Rolling update. The planned v1 → v2 rollout was not performed, because v2 serves empty predictions (Task 2) and a rollout to it would have served a broken model; the deployment was instead rolled back to v1 by rollout restart, which is the state now running. This is a genuine limitation of the project, not a completed experiment. Single-node k3d does not prove multi-machine fault tolerance; all of the above was measured on one node, k3d-cs3081-server-0.

5 Networking (Task 5)

Subnets were carved from 10.20.0.0/16, largest first (usable = 2^(32−prefix) − 2):

Subnet	Range	Usable	Broadcast	Hosts
app	10.20.0.0/22	10.20.0.1–10.20.3.254	10.20.3.255	1,022
public	10.20.4.0/24	10.20.4.1–10.20.4.254	10.20.4.255	254
data	10.20.5.0/25	10.20.5.1–10.20.5.126	10.20.5.127	126
management	10.20.5.128/27	10.20.5.129–10.20.5.158	10.20.5.159	30

The network ACL (network/nacl.json) denies first, then allows: inbound rule 100 denies all traffic from 198.51.100.0/24 (a lower number than the allows, so it wins), 110 allows TCP 80 from anywhere, 120 allows TCP 443 from anywhere, 130 allows TCP 22 only from management 10.20.5.128/27, 140 allows TCP 1024–65535 from app 10.20.0.0/22 (replies); outbound 100 allows TCP 8000 to app, 110 allows TCP 1024–65535 anywhere (replies). The NACL is stateless, so replies need their own rules, and rule order (lowest number first) is what makes the deny work. Verified 6/6 with tools/nacl_sim.py --check on the six sample packets.

6 Reproducibility and CI (Task 6)

The repository is https://github.com/Rishi-Ra-1j/Cloud_Project, branch main, commit d974839, 70 files, 6,055 insertions. Models, .venv, .s3-credentials, and the kit zip were excluded from Git (.gitignore); the model is a Release asset, never committed. *make up* (git clone, cd, make up — three commands) creates the cluster,

SeaweedFS, and buckets; *make down* deletes the cluster. GitHub Actions (.github/workflows/ci.yml) runs, in order, lint (ruff + py_compile), 11 unit tests (test_clean.py 6, test_shard.py 2, test_schema.py 3 — 11 passed locally), evaluation (downloads pythia-70m-v1.zip from the Release, runs scripts/evaluate.py student, then the gate tools/score.py --min-validity 95), then gitleaks. The Release is v1 at https://github.com/Rishi-Ra-1j/Cloud_Project/releases/tag/v1, asset pythia-70m-v1.zip (217 MB, model + tokenizer + model.sha256). The deliberately failed gate run is the untrained base model at 0% validity. AI_USE.md discloses the drafting help used, that teacher replies were never edited, and that every report number comes from a log, score.py, kubectl, or a Locust CSV. The final commit is tagged **final** after this report is added.

7 Load, cost, and review (Task 7)

The standard Locust run (4 users, 2 minutes, host parse.local:8080 — the localhost host gave 229,882 requests in 2 ms, all HTTP 404, because the Ingress only answers for parse.local, so parse.local was added to /etc/hosts) gave 12 requests, 12 failed (100%), all timing out at 30,000 ms, HTTP 0. A 1-user run gave 4 requests, 3 failed on timeout, 1 succeeded in 28,980 ms (served by mkhh5, v1), throughput 0.03 req/s, p50/p95 30,000 ms (timeout-capped). The service is correct but too slow for Locust's 30-second timeout: one parse takes 26–29 s on CPU, so under any concurrency it exceeds the timeout. That is the finding; it is not a tuning problem that was fixed in the time available.

Cost (scripts/cost.py on the 1-user run, rate_card.json: vCPU ■3.50/hour, GiB ■0.45/hour, billed on limits): per replica 1 × ■3.50 + 1 × ■0.45 = ■3.95/hour; 2 replicas = ■7.90/hour; 0.03 req/s × 3600 = 121 requests/hour; cost per 1,000 requests = ■7.90 / 121 × 1000 = **■65.29**. The cost is high because throughput is low, not because the hourly price is high. SLO: 99% of /parse requests return HTTP 200 over the 2-minute window. Error budget = 1% of 4 requests = 0.0 requests; budget used = 3 failures, 7500% of budget — the SLO is breached. Full output is in results/cost.json.

Aspect	Measured	Evidence
JSON validity (v1)	93.0%	results/student_validation.json (clean extraction)
Full frame (v1 / teacher)	35.0% / 37.0%	results/student_validation.json, results/teacher_validation.json
Load balancing	5 / 5 (50/50)	X-Served-By counts, 10 parallel requests
Cold start (1 run)	102 s	Pod delete 09:28:32 → Ready 09:30:14
Peak pod memory	724 Mi	kubectl top pods -n app (limit 1 Gi)
Locust (1 user)	1/4 succeeded, 28,980 ms	results/loadtest1_stats.csv
Cost per 1,000	■65.29	results/cost.json

Conclusion. The distillation works: a 70M student trained only on teacher-generated replies reaches 35% full-frame accuracy against the teacher's 37%, with 93% JSON validity, and serves correctly as a Kubernetes service with two replicas, load balancing, and a 102-second cold start. The two honest failures are the masked v2 training (0%, first-token prediction broken) and the load-test latency: at 26–29 s per parse on CPU, the service cannot meet a 30-second timeout under load, which also makes the cost per 1,000 requests high and breaches the 99% SLO. The next steps would be to reduce generated tokens or use a faster runtime for serving, and to retrain v2 as whole-sequence training with EOS instead of masked training.

References. Bastianelli et al., SLURP (EMNLP 2020), CC BY 4.0 (commands). Phi-3.5-mini-instruct and EleutherAI/pythia-70m model cards. All code, manifests, tests, CI, and results are in the GitHub repository above; the model is in Release v1.